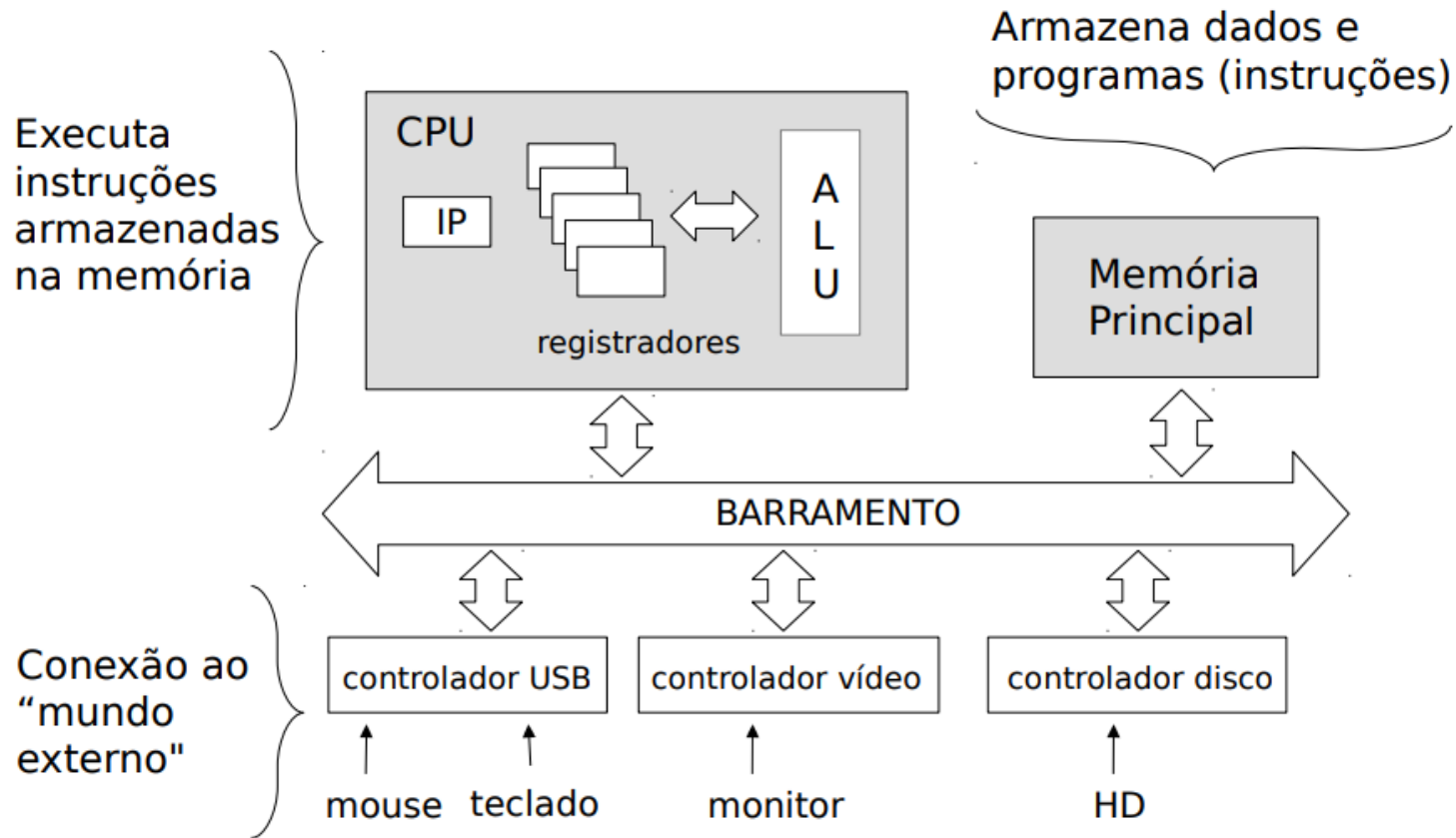


Software Básico

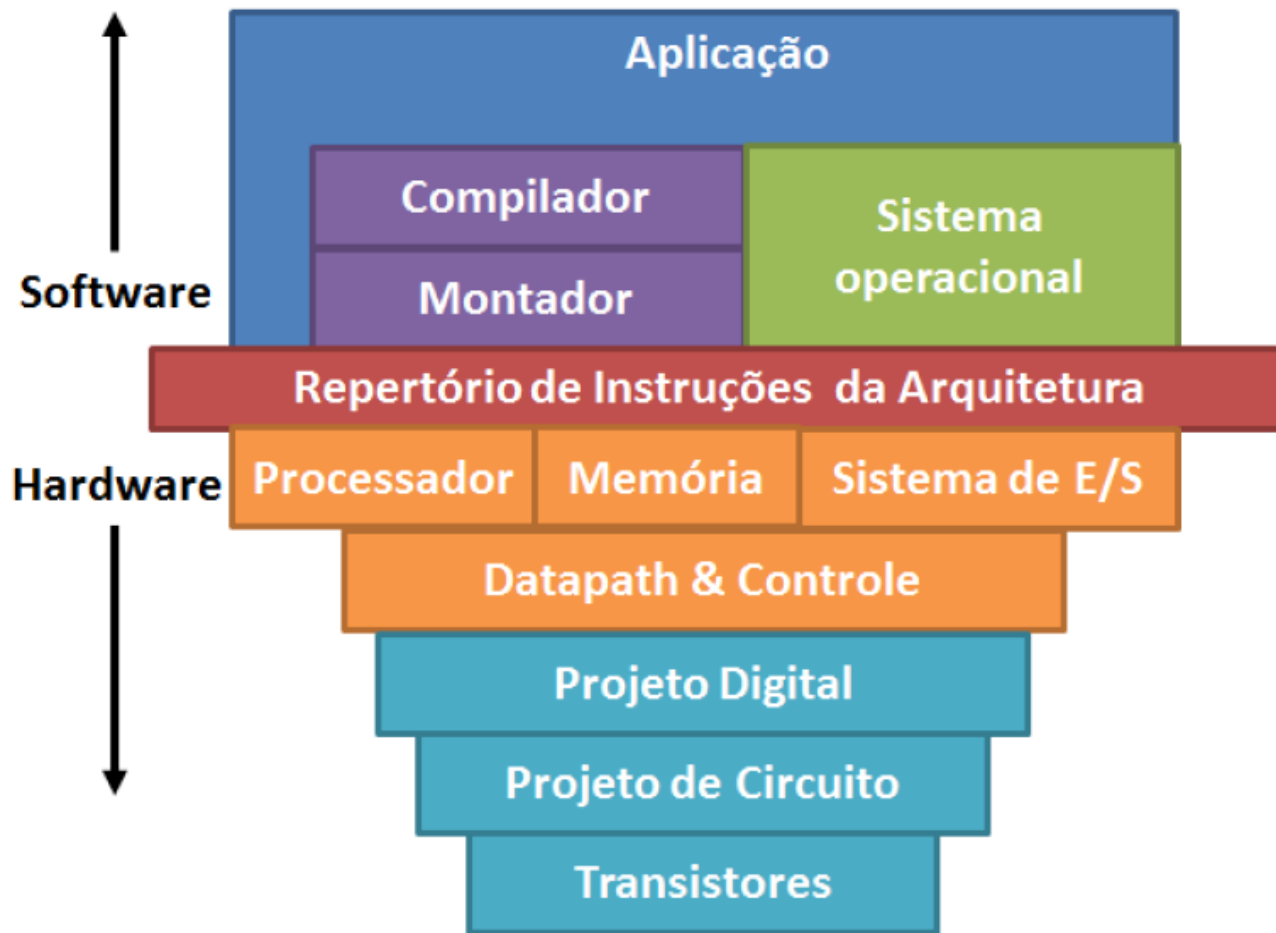


**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
NORTE DE MINAS GERAIS**

Arquitetura



Sistema de Computação



Software Básico

- Todo programa necessário para o funcionamento do computador;
- É um conjunto de programas que define o padrão de comportamento do equipamento, tornando-o utilizável, ou seja, são os programas usados para permitir o funcionamento do *hardware*.



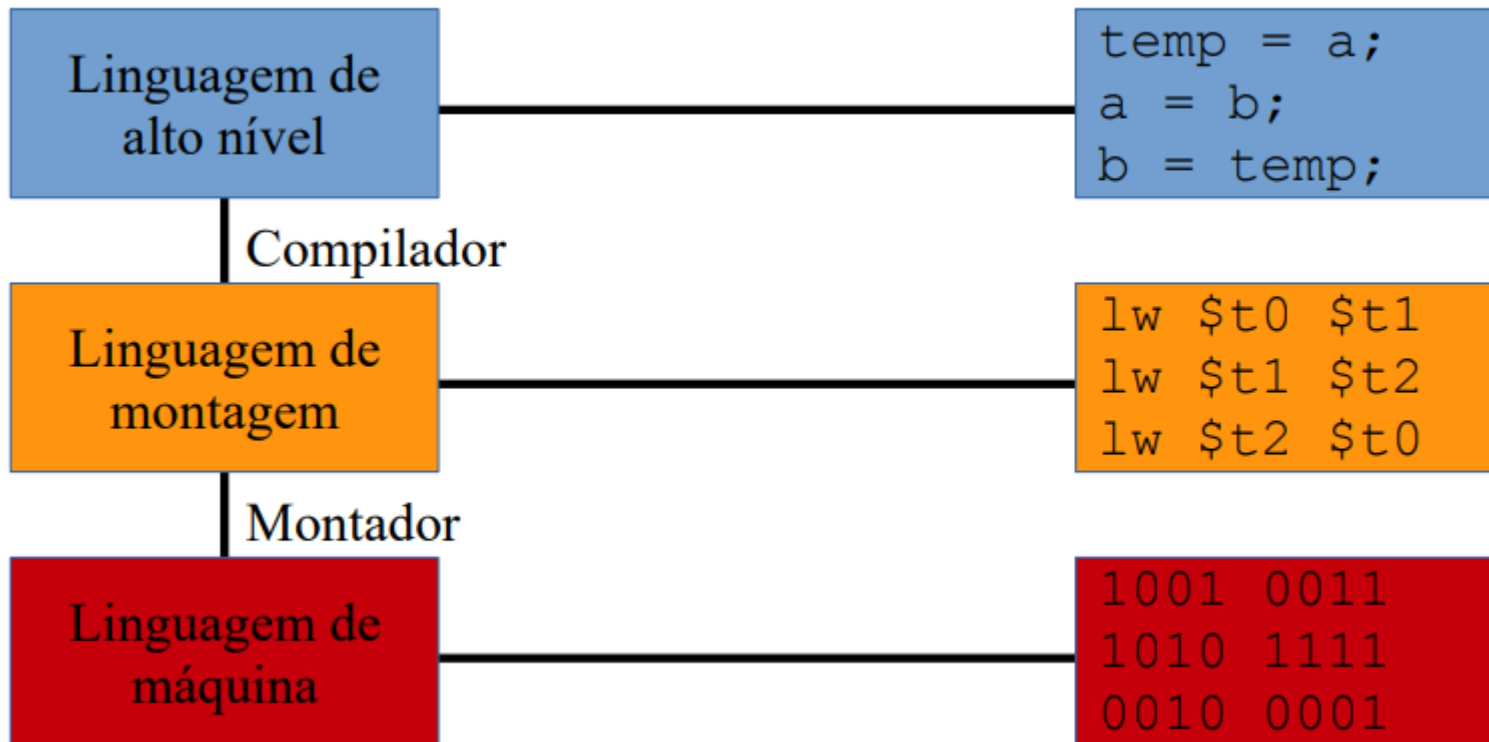
Sistema de Computação

- Necessário uma organização estruturada de computadores para facilitar a comunicação homem-computador.
 - **Solução:** definir uma hierarquia de ***abstrações de níveis mais alto*** baseados nos níveis mais baixos.
 - Cada nível depende apenas de uma caracterização abstrata e simplificada do sistema de nível imediatamente inferior.
 - Abstraindo os conceitos de *hardware*.
 - Criando linguagens de programação;



Linguagens x Níveis

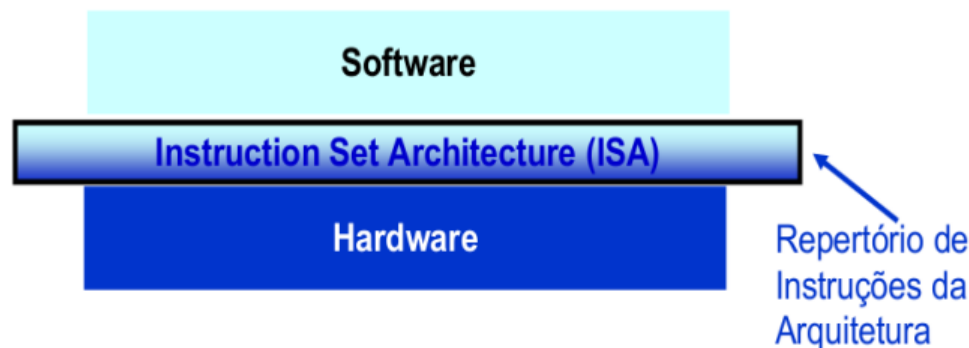
As linguagens de programação variam de acordo com o seu nível de abstração.



Interface *Software/Hardware*

Conjunto de instruções: Provê a informação necessária para que se escreva um código em linguagem de máquina (ou montagem) que execute corretamente na arquitetura.

- Acesso a memória;
- Instruções;
- Registradores;
- Entrada/Saída;



Interface *Software/Hardware*

- As linguagens de programação podem ser classificadas em 6 níveis:
 - Nível lógico digital (Nível 0);
 - Nível de microarquitetura (Nível 1) ;
 - Nível de conjunto de instruções (Nível 2) ;
 - Nível de sistema operacional (Nível 3);
 - Nível de montagem (Nível 4);
 - Nível de Aplicação (Nível 5);



Interface *Software/Hardware*

- As linguagens de programação podem ser classificadas em 6 níveis:
 - **Nível lógico digital (Nível 0):**
 - Os objetos de interesse são conhecidos como **portas lógicas**;
 - Cada **porta lógica** possui 1 ou mais entradas digitais (aceitam 0 ou 1) e calculam funções lógicas simples sobre essas entradas:
 - AND, OR, XOR, ...



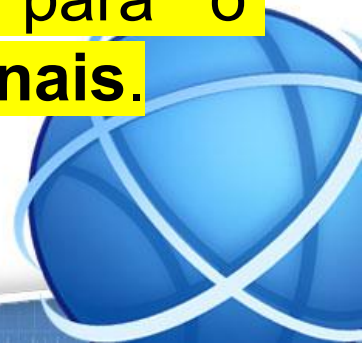
Interface *Software/Hardware*

- As linguagens de programação podem ser classificadas em 6 níveis:
 - **Nível de microarquitetura (Nível 1):**
 - Uma memória local (**registradores**) e uma **ULA** (Unidade Lógica e Aritmética) que realiza operações aritméticas muito simples.
 - O **caminho dos dados** é formado pela conexão dos registradores com a ULA.
 - **Controle:** microprograma ou direto pelo *hardware* (**Unidade de Controle**);



Interface *Software/Hardware*

- As linguagens de programação podem ser classificadas em 6 níveis:
 - **Nível de conjunto de instruções (Nível 2):**
 - Nível ISA (*Instruction Set Architecture*).
 - Definida pelo fabricante e depende da **arquitetura da máquina**.
 - O fabricante disponibiliza o manual de referência da linguagem de máquina;
 - Essas informações são necessárias para o desenvolvedores de **sistemas operacionais**.



Interface *Software/Hardware*

- As linguagens de programação podem ser classificadas em 6 níveis:
 - **Nível de sistema operacional (Nível 3):**
 - Contém quase todas as instruções do nível ISA, bem como o conjunto de novas instruções que o sistema operacional adiciona;
 - Novas instruções – ***chamadas de sistema***;



Interface *Software/Hardware*

- As linguagens de programação podem ser classificadas em 6 níveis:
 - **Nível de montagem (Nível 4):**
 - A linguagem de montagem é uma forma simbólica de representação das linguagens do nível mais baixo.
 - Programas em linguagem de montagem são, inicialmente, traduzidos para as linguagens dos níveis 1, 2 e 3 e depois interpretados pela máquina.
 - O programa que realiza essa tradução é chamado de **montador**.

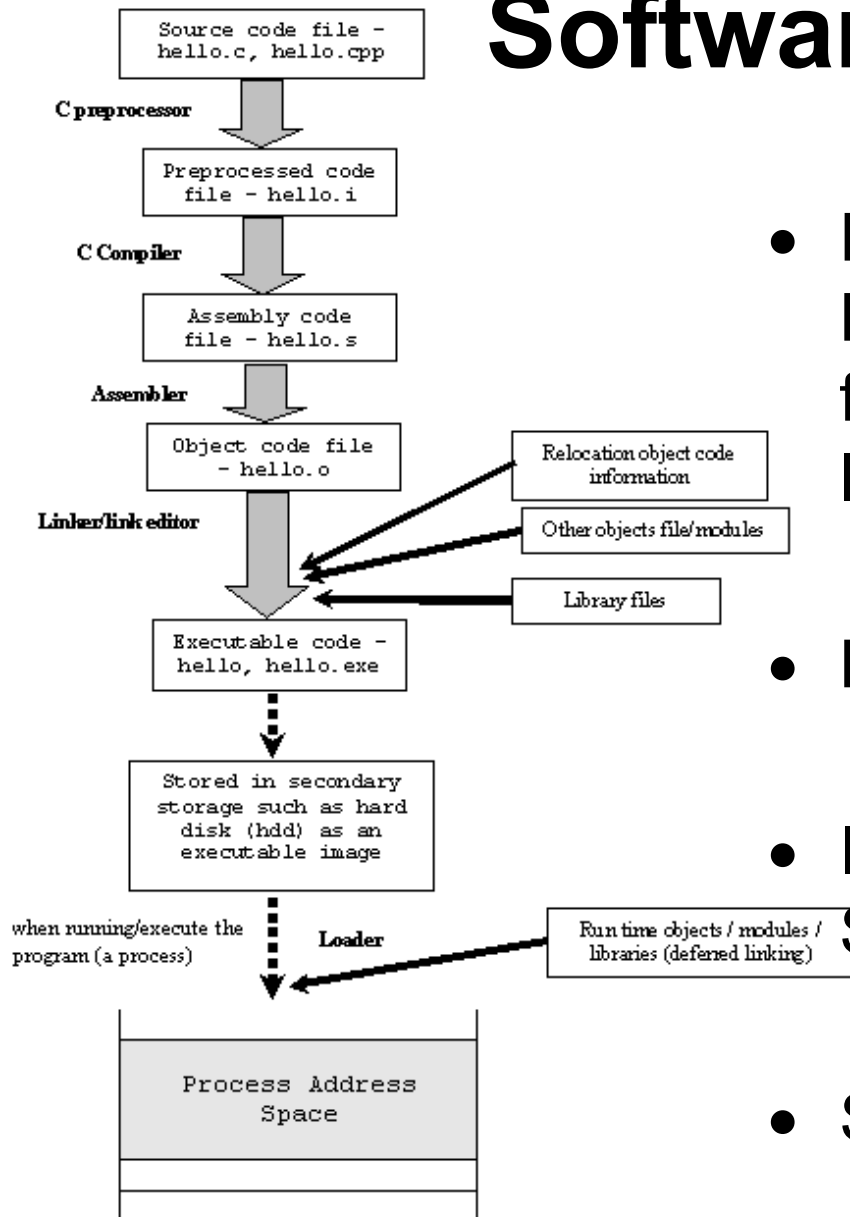


Interface *Software/Hardware*

- As linguagens de programação podem ser classificadas em 6 níveis:
 - **Nível de Aplicação (Nível 5):**
 - Nesse nível as linguagens são conhecidas como linguagens de alto nível.
 - C, C++, C# , Java, ~~Python~~, ...
 - Os programas são geralmente traduzidos para o nível 3 e 4 por **compiladores**.
 - Alguns desses programas podem ser **interpretados**, como os programas em Java.

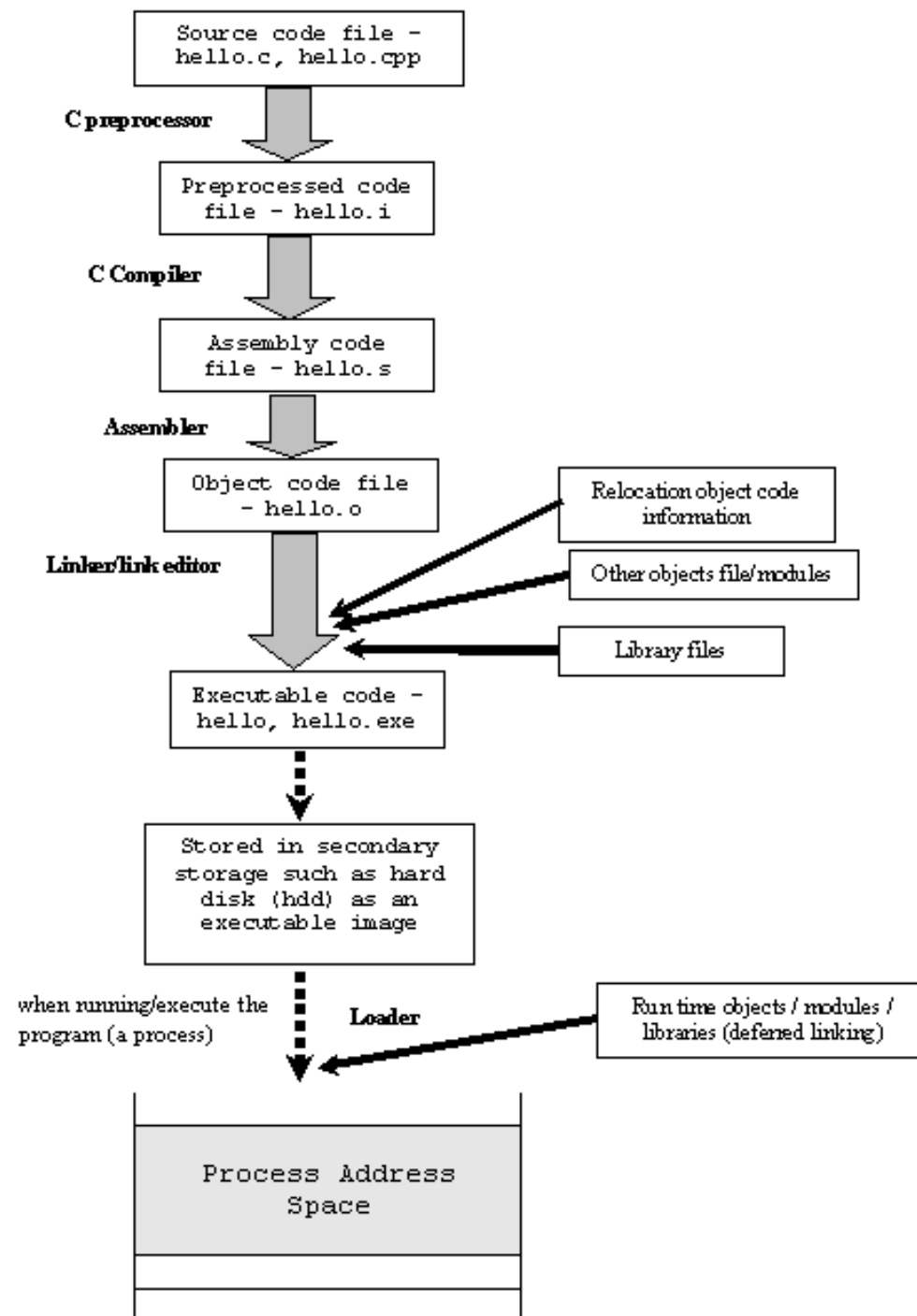


Software Básico



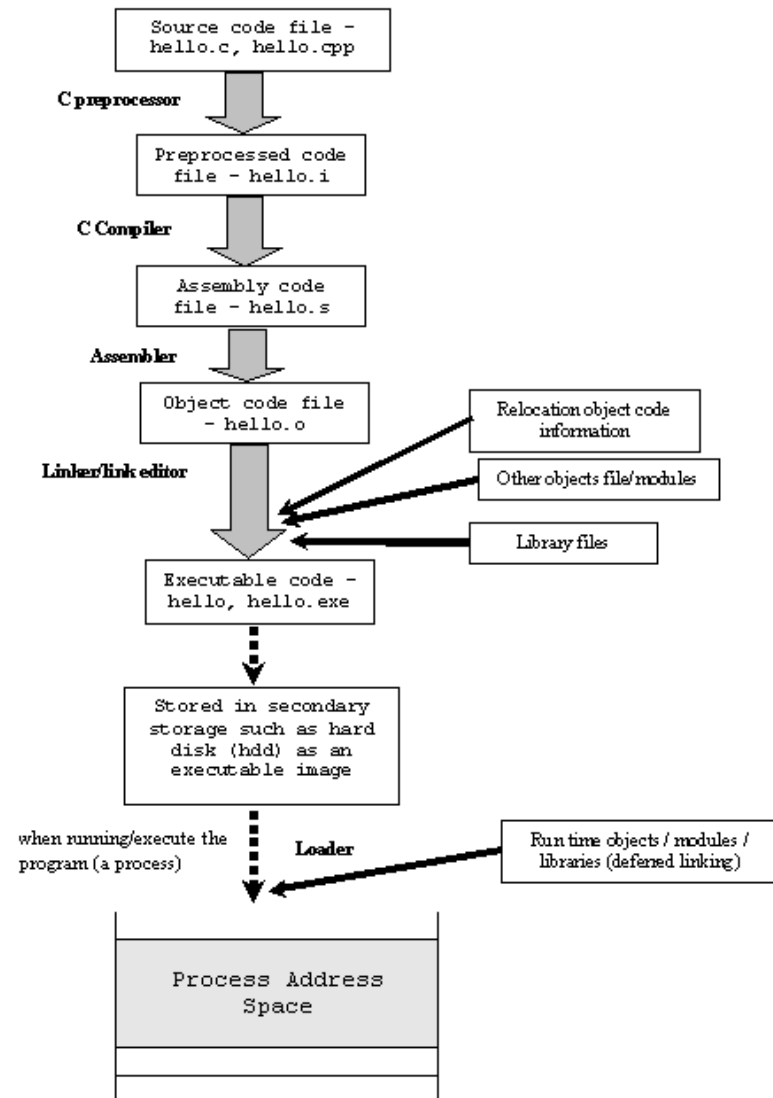
- Representação de dados, Mecanismos de controle, funções, pilha, variáveis locais;
- Linguagem de montagem;
- Interrupções, Interação com S.O;
- Softwares básicos;





Software Básico

- Todo programa necessário para o funcionamento do computador;
- É um conjunto de programas que define o padrão de comportamento do equipamento, tornando-o utilizável, ou seja, são os programas usados para permitir o funcionamento do *hardware*.



Programa armazenado

Programa
em linguagem
de alto nível
(em C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compilador

Programa
em linguagem
assembly
(para o MIPS)

```
swap:
    multi $2, $5,4
    add   $2, $4,$2
    lw    $15, 0($2)
    lw    $16, 4($2)
    sw    $16, 0($2)
    sw    $15, 4($2)
    jr    $31
```

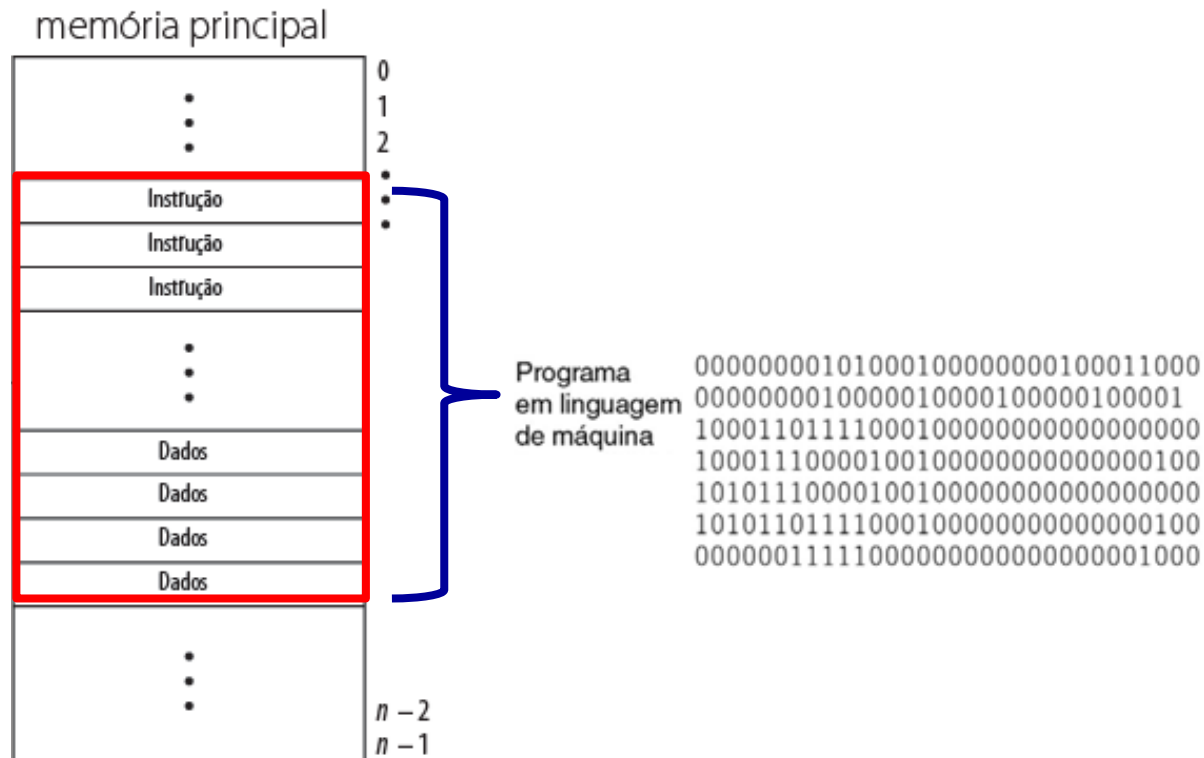
Assembler

Programa
em linguagem
de máquina
(para o MIPS)

```
000000001010001000000000100011000
0000000010000010000100000100001
10001101111000100000000000000000
100011100001001000000000000000100
101011100001001000000000000000000
101011011110001000000000000000100
00000011111000000000000000001000
```



Programa armazenado



Armazenamento

- As linguagens de programação possuem variáveis simples, que contêm elementos de dados isolados, mas também possuem estruturas de dados mais complexas.
- Essas estruturas de dados complexas podem conter muito mais elementos de dados do que a quantidade de registradores em um computador.



Memória Principal

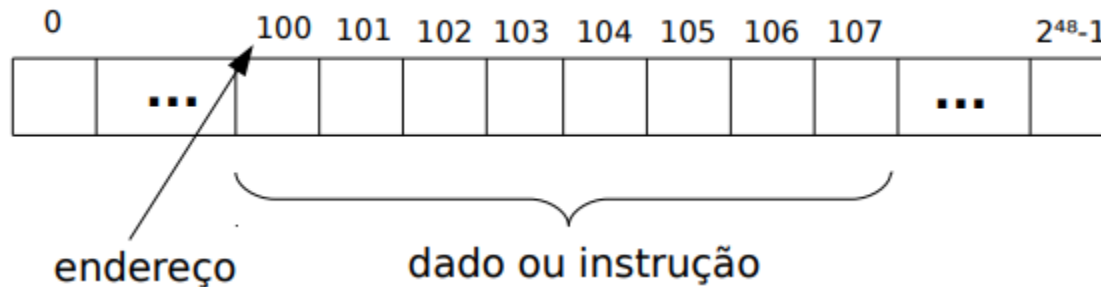
- O processador só pode manter uma pequena quantidade de dados nos registradores;
- A **memória do computador** contém milhões de elementos de dados. Logo, as estruturas de dados (*arrays* e estruturas) são mantidas na memória;

A memória é apenas uma **sequência grande e unidimensional**, com o **endereço** atuando como índice para esse *array*, começando de 0.



Memória Principal

- **Armazena dados e instruções:**
 - dados ocupam um número de bytes que depende de seu tipo;
 - instruções ocupam um número variável de bytes;



Representação de dados

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

(32 bits de largura)

- Os números são mantidos no *hardware* do computador como uma série de sinais eletrônicos altos e baixos, e por isso são considerados números de base 2.
- Os *bits* são numerados com 0, 1, 2, 3, ... da direita para a esquerda em uma **palavra**.



Representação de dados

- O termo:
 - ***bit* menos significativo** é usado para se referir ao *bit* mais à direita e
 - ***bit* mais significativo** para o *bit* mais à esquerda (*bit* 31).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MSB																								LSB							



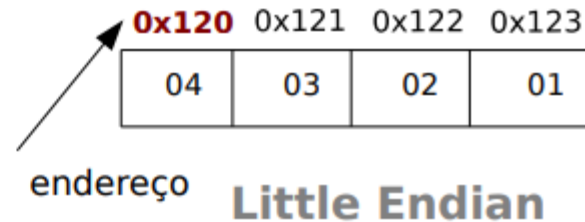
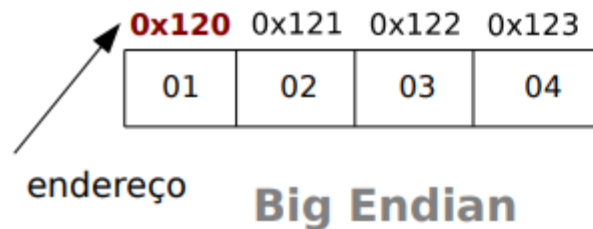
Representação de dados

- Valores inteiros em diferentes tamanhos (número de *bytes*)
 - com 1 byte (8 *bits*) representamos inteiros de 0 a 255 (2^8-1);
 - com 2 bytes (16 *bits*), de 0 a 65535 ($2^{16}-1$);
 - com 4 bytes (32 *bits*), de 0 a 4294967295 ($2^{32}-1$);
 - com 8 bytes (64 *bits*), de 0 a $2^{64}-1$;
- A mesma limitação vale para **endereços**:
 - com 4 *bytes* podemos endereçar 4GB de MP;
 - com 8 *bytes* podemos endereçar (teoricamente) 2^{64} *bytes*;



Representação de dados

- Duas convenções para ordem de armazenamento dos *bytes* na memória:
 - **Big Endian** (IBM System 370/390, Motorola 680x0, Sun SPARC): do *byte* mais significativo para o menos significativo;
 - **Little Endian** (Intel 80x86, x86, VAX e Alpha): do *byte* menos significativo para o mais significativo

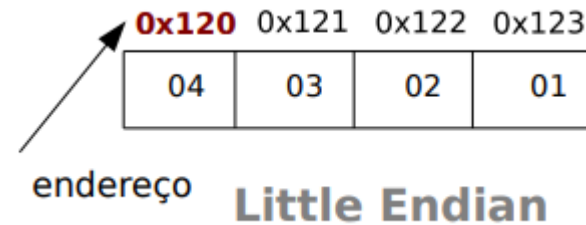


0x01020304



Representação de dados

- **Little Endian** (Intel): do *byte* menos significativo para o mais significativo:



```
#include <stdio.h>

void dump (void *p, int n)
{
    unsigned char *p1 = p;
    while (n--)
    {
        printf("%p - %02x\n", p1, *p1);
        p1++;
    }
}

int main()
{
    int num = 0x01020304;

    dump(&num, sizeof(int));

    return 0;
}
```

```
"D:\IFNMG\Professor\Disciplinas\2021-2 ANP\SB\Aulas\Semana 1\exem -"
0060FEFC - 04
0060FEFD - 03
0060FEFE - 02
0060FEFF - 01

Process returned 0 (0x0)   execution time : 0.228 s
Press any key to continue.
```



Representação de dados

Endereço de byte Mapeamento de endereço big-endian

	11	12	13	14				
00	00	01	02	03	04	05	06	07
08	21	22	23	24	25	26	27	28
	08	09	0A	0B	0C	0D	0E	0F
10	31	32	33	34	'A'	'B'	'C'	'D'
	10	11	12	13	14	15	16	17
18	'E'	'F'	'G'		51	52		
	18	19	1A	1B	1C	1D	1E	1F
20	61	62	63	64				
	20	21	22	23				

00	11
	12
	13
	14
04	
08	21
	22
	23
	24
0C	25
	26
	27
	28
10	31
	32
	33
	34
14	'A'
	'B'
	'C'
	'D'
18	'E'
	'F'
	'G'
1C	51
	52
20	61
	62
	63
	64

(a) Big-endian

00	14
	13
	12
	11
04	
08	28
	27
	26
	25
0C	24
	23
	22
	21
10	34
	33
	32
	31
14	'A'
	'B'
	'C'
	'D'
18	'E'
	'F'
	'G'
1C	52
	51
20	64
	63
	62
	61

(b) Little-endian

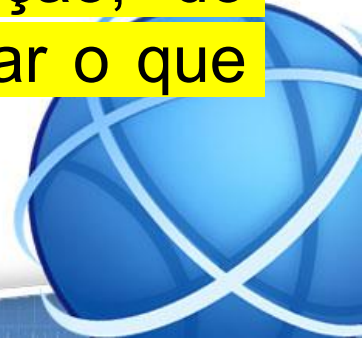
Endereço de byte Mapeamento de endereço little-endian

	07	06	05	04	11	12	13	14
00	07	06	05	04	03	02	01	00
08	21	22	23	24	25	26	27	28
	0F	0E	0D	0C	0B	0A	09	08
10	'D'	'C'	'B'	'A'	31	32	33	34
	17	16	15	14	13	12	11	10
18			51	52		'G'	'F'	'E'
	1F	1E	1D	1C	1B	1A	19	18
20					61	62	63	64
					23	22	21	20



Representação de dados

- O *hardware* pode ser projetado para **somar, subtrair, multiplicar e dividir** esses padrões de *bits*.
- Se o número que é o resultado correto de tais operações **não** puder ser representado por esses *bits* de *hardware* mais à direita, diz-se que houve um **overflow**.
- Fica a critério da linguagem de programação, do sistema operacional e do programa determinar o que fazer quando isso ocorre.



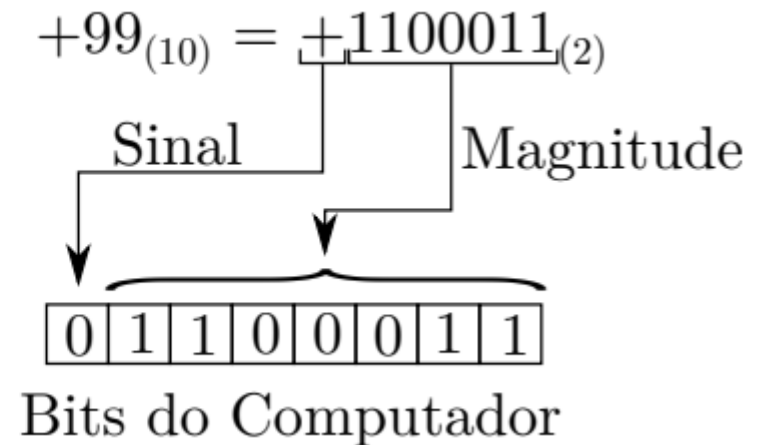
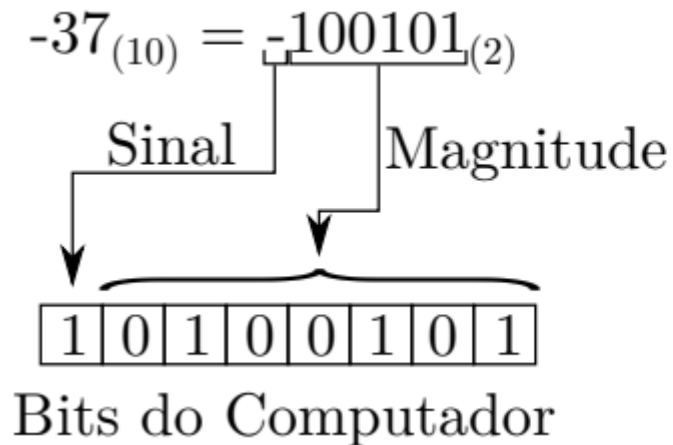
Representação de dados

- Os programas de computador calculam números **positivos** e **negativos**, de modo que precisamos de uma **representação** que faça a distinção entre o positivo e o negativo.
- A solução mais simples é acrescentar um sinal separado, que convenientemente possa ser representado em um **único *bit***; o nome dessa representação é **sinal e magnitude**.



Representação de dados

- **Sinal Magnitude:** *bit de sinal*



Representação de dados

- Outra alternativa de representação 0s iniciais significa positivo e 1s iniciais significa negativo. Essa convenção para representar os números binários com sinal é chamada representação por **complemento de**

dois: 0000 0000 0000 0000 0000 0000 0000 0000_{bin} = 0_{dec}
0000 0000 0000 0000 0000 0000 0000 0001_{bin} = 1_{dec}
0000 0000 0000 0000 0000 0000 0000 0010_{bin} = 2_{dec}
... ..

0111 1111 1111 1111 1111 1111 1111 1101_{bin} = 2.147.483.645_{dec}
0111 1111 1111 1111 1111 1111 1111 1110_{bin} = 2.147.483.646_{dec}
0111 1111 1111 1111 1111 1111 1111 1111_{bin} = 2.147.483.647_{dec}
1000 0000 0000 0000 0000 0000 0000 0000_{bin} = -2.147.483.648_{dec}
1000 0000 0000 0000 0000 0000 0000 0001_{bin} = -2.147.483.647_{dec}
1000 0000 0000 0000 0000 0000 0000 0010_{bin} = -2.147.483.646_{dec}
... ..

1111 1111 1111 1111 1111 1111 1111 1101_{bin} = -3_{dec}
1111 1111 1111 1111 1111 1111 1111 1110_{bin} = -2_{dec}
1111 1111 1111 1111 1111 1111 1111 1111_{bin} = -1_{dec}



Representação de dados

- A representação por **complemento de dois** tem a vantagem de que todos os números negativos possuem 1 no *bit* mais significativo.
- o *hardware* só precisa testar esse *bit* para ver se um número é positivo ou negativo (com 0 considerado positivo).
 - Os programas desejam lidar às vezes com números que podem ser positivos ou negativos e às vezes com números que só podem ser positivos.
 - Os endereços de memória começam com 0 e continuam até o maior endereço. Os endereços negativos não fazem sentido.



Representação de dados

- Codificação sem sinal (***unsigned***):
 - O **sistema de numeração binária** é usado, de forma direta, para codificar os números naturais ou os números inteiros sem sinal. Com n *bits* podemos representar, de forma não ambígua, até 2^n valores (0 até 2^n-1).
- Codificação em complemento de 2 (***signed***):
 - Com n *bits* nesta notação podemos representar os valores de -2^{n-1} até $2^{n-1}-1$.



Representação de dados

- **Intervalos de valores (complemento de dois):**
 - Com n bits:
 - O menor valor representado e -2^{n-1}
 - O maior valor representado e $2^{n-1} - 1$
 - Com 8 bits $\rightarrow [-2^7, 2^7 - 1] \rightarrow [-128, 127]$
 - Com 16 bits $\rightarrow [-2^{15}, 2^{15} - 1] \rightarrow [-32768, 32767]$



Representação de dados

- Exemplos 16 bits:

	Decimal	Hex	Bin
UMax	65535	FF FF	11111111 11111111
TMax	32767	7F FF	01111111 11111111
TMin	-32768	80 00	10000000 00000000
-1	-1	FF FF	11111111 11111111
0	0	00 00	00000000 00000000

- inteiros sem sinal (***unsigned***): ocupam o mesmo tamanho que os tipos com sinal, mas representam um intervalo diferente de valores;



Representação de dados

Tipos de dados inteiros em máquinas de 32 *bits*

C data type	Minimum	Maximum
char	-128	127
unsigned char	0	255
short [int]	-32,768	32,767
unsigned short [int]	0	65,535
int	-2,147,483,648	2,147,483,647
unsigned [int]	0	4,294,967,295
long [int]	-2,147,483,648	2,147,483,647
unsigned long [int]	0	4,294,967,295
long long [int]	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long long [int]	0	18,446,744,073,709,551,615



Representação de dados

Tipos de dados inteiros em máquinas de 64 *bits*

C data type	Minimum	Maximum
char	-128	127
unsigned char	0	255
short [int]	-32,768	32,767
unsigned short [int]	0	65,535
int	-2,147,483,648	2,147,483,647
unsigned [int]	0	4,294,967,295
long [int]	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long [int]	0	18,446,744,073,709,551,615
long long [int]	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long long [int]	0	18,446,744,073,709,551,615



Representação de dados

- *signed* x *unsigned*

```
#include <stdio.h>

int main()
{
    short int sv = -12345;
    unsigned short uv = (unsigned short) sv;

    printf("sv = %d, uv = %u\n", sv, uv);

    return 0;
}
```

```
sv = -12345, uv = 53191
```

```
Process returned 0 (0x0)   execution time : 0.203 s
Press any key to continue.
```


Representação de dados

- *signed* x *unsigned*

```
#include <stdio.h>

int main()
{
    unsigned u = 4294967295u; /* UMax_32 */
    int tu = (int) u;

    printf("u = %u, tu = %d\n", u, tu);

    return 0;
}
```

```
u = 4294967295, tu = -1
```

```
Process returned 0 (0x0)   execution time : 0.160 s
Press any key to continue.
```

Representação de dados

- *signed* x *unsigned*

```
#include <stdio.h>

int main()
{
    unsigned char u = 255;
    int tu = (int) u;

    printf("u = %u, tu = %d\n", u, tu);

    return 0;
}
```

u = 255, tu = 255

Process returned 0 (0x0) execution time : 0.156 s
Press any key to continue.

Representação de dados

- *signed x unsigned* em C:

```
int a[2] = {-1, 0};  
unsigned int z = 0;
```

```
if (a[0] < a[1]) /* true */  
if (a[0] < z)    /* false */  
if (a[0] < 0)    /* true */  
if (a[0] < 0U)   /* false */
```



Representação de dados

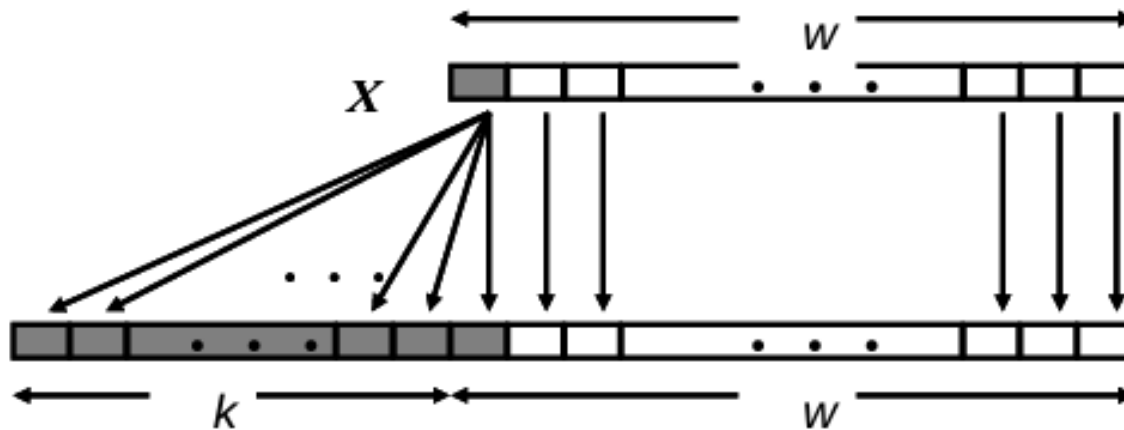
- *signed x unsigned* em C:

Expression	Type	Evaluation
0 == 0U	unsigned	1
-1 < 0	signed	1
-1 < 0U	unsigned	0 *
2147483647 > -2147483647-1	signed	1
2147483647U > -2147483647-1	unsigned	0 *
2147483647 > (int) 2147483648U	signed	1 *
-1 > -2	signed	1
(unsigned) -1 > -2	unsigned	1



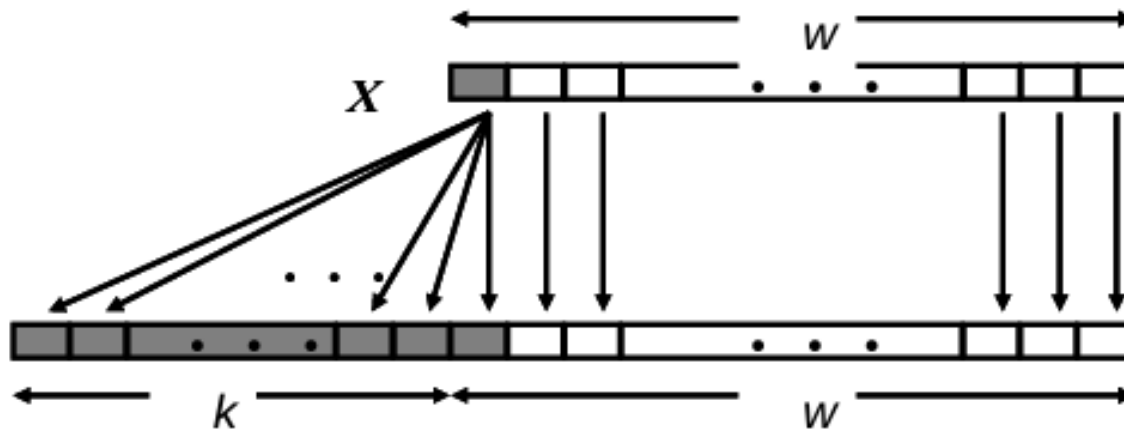
Extensão de Representação

- Conversões que aumentam o tamanho:
char para short, short para int, int para long, ...
- Converter representação com w bits para uma representação com $w+k$ bits, mantendo o valor **mais significativo** de w :



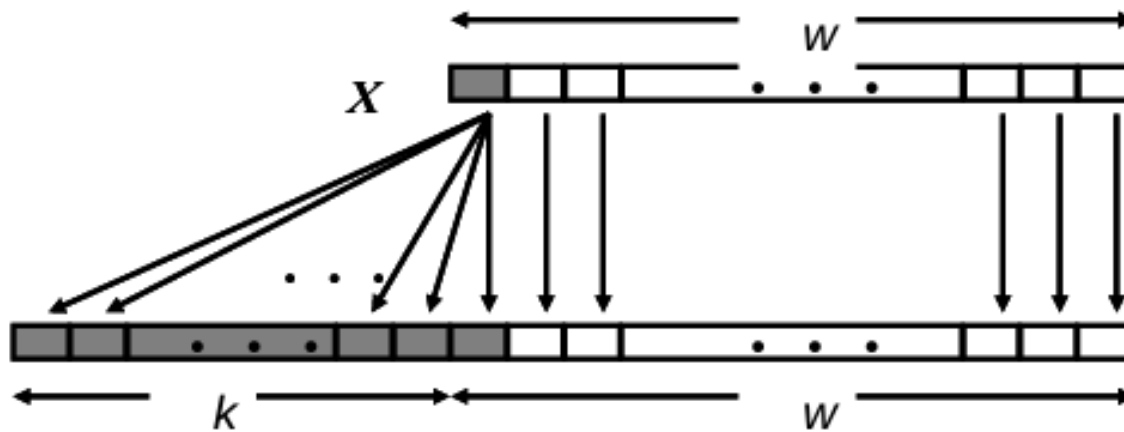
Extensão de Representação

- Extensão sem sinal (**zero extension**) adicionar k bits 0 à esquerda:



Extensão de Representação

- Extensão sem sinal (**zero extension**) adicionar k *bits* 0 à esquerda.
- Extensão para **complemento a 2** (**sign extension**) adicionar k cópias do *bit* mais significativo à esquerda.



Representação de dados

```
1  short sx = -12345;           /* -12345 */
2  unsigned short usx = sx;     /* 53191 */
3  int x = sx;                  /* -12345 */
4  unsigned ux = usx;           /* 53191 */
5
6  printf("sx = %d:\t", sx);
7  show_bytes((byte_pointer) &sx, sizeof(short));
8  printf("usx = %u:\t", usx);
9  show_bytes((byte_pointer) &usx, sizeof(unsigned short));
10 printf("x = %d:\t", x);
11 show_bytes((byte_pointer) &x, sizeof(int));
12 printf("ux = %u:\t", ux);
13 show_bytes((byte_pointer) &ux, sizeof(unsigned));
```



Representação de dados

```
#include <stdio.h>
2
3 typedef unsigned char *byte_pointer;
4
5 void show_bytes(byte_pointer start, int len) {
6     int i;
7     for (i = 0; i < len; i++)
8         printf(" %.2x", start[i]);
9     printf("\n");
10 }
11
```



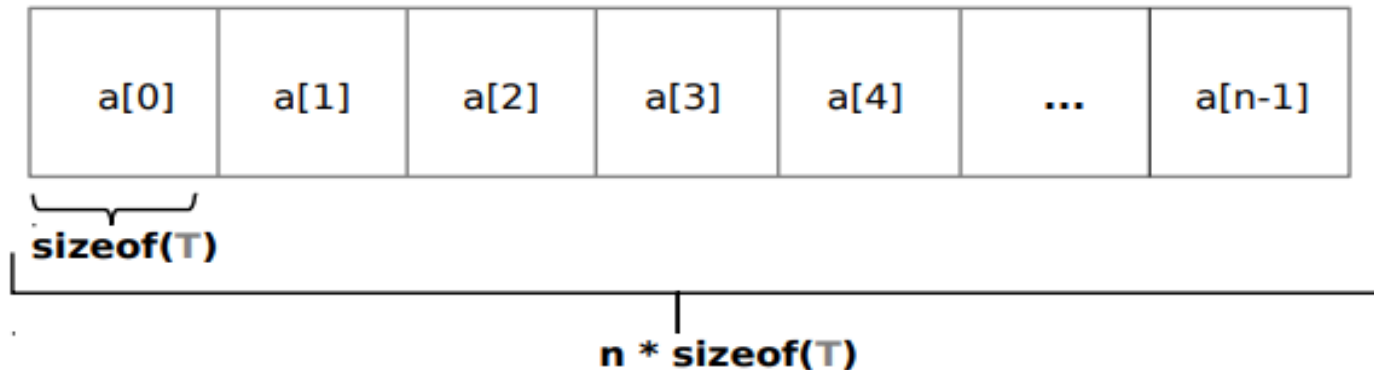
Representação de dados

```
12 void show_int(int x) {  
13     show_bytes((byte_pointer) &x, sizeof(int));  
14 }  
15  
16 void show_float(float x) {  
17     show_bytes((byte_pointer) &x, sizeof(float));  
18 }  
19  
20 void show_pointer(void *x) {  
21     show_bytes((byte_pointer) &x, sizeof(void *));  
22 }
```



Representação de estruturas homogêneas de dados

- C usa uma implementação bastante simples:
 - alocação contígua na memória;

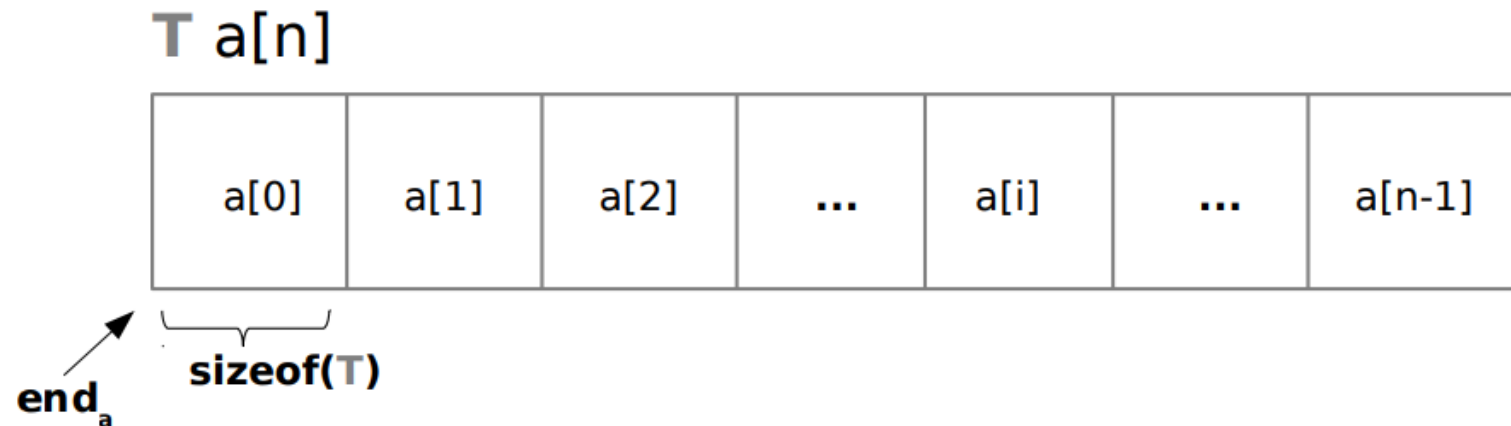


- Para um tipo **T** e uma constante **n**, a declaração **T a[n]** aloca uma região contígua de memória com tamanho igual a **n * sizeof(T) bytes**;



Representação de estruturas homogêneas de dados

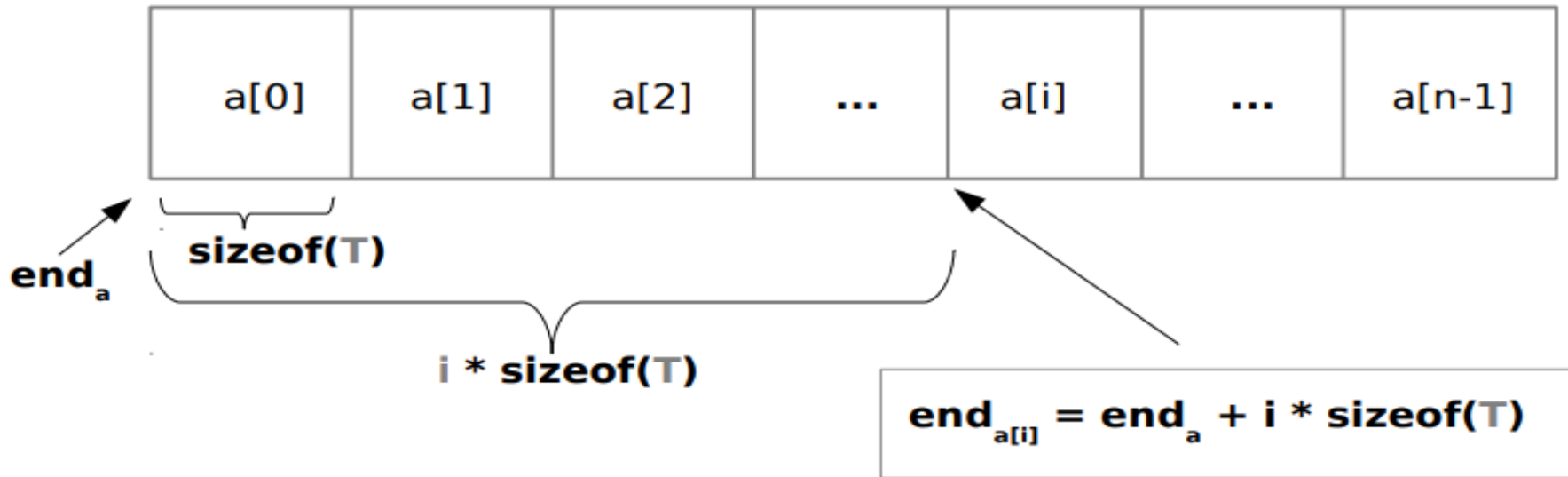
- Endereços dos Elementos



Representação de estruturas homogêneas de dados

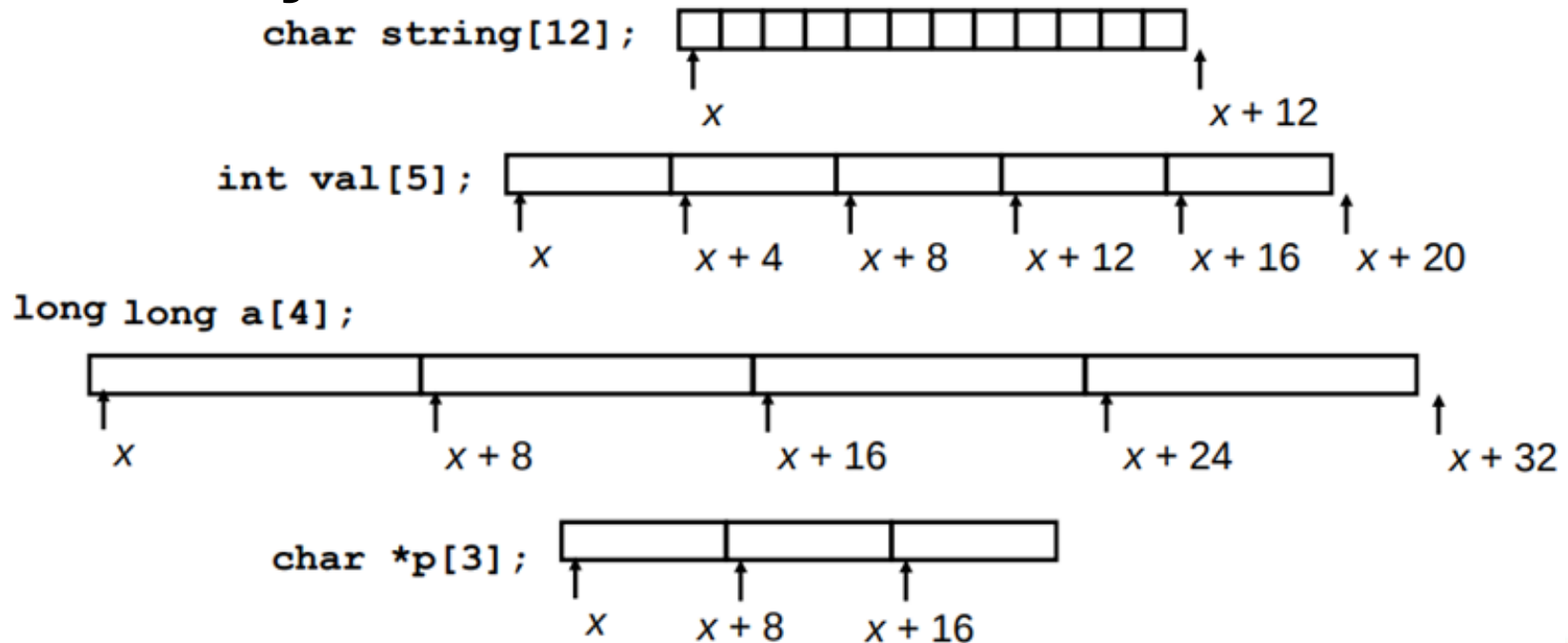
- Endereços dos Elementos

T a[n]



Representação de estruturas homogêneas de dados

- Endereços dos Elementos



Representação de estruturas homogêneas de dados

- Relação entre Ponteiros e *Arrays*:
 - O nome de um *array* equivale a um ponteiro (constante) para o tipo de seus elementos;
 - A declaração `int a[N]`
 - `a` é um ponteiro constante do tipo `int *` e seu valor é `&a[0] = a = &a`
 - Ao passarmos um *array* como **parâmetro**, passamos seu endereço (i.e., um ponteiro para seu primeiro elemento) ;



Representação de estruturas homogêneas de dados

- Relação entre Ponteiros e *Arrays*:
 - C permite operações aritméticas básicas com ponteiros;

```
int a[5];  
int *p = a
```

} $a[3] \Leftrightarrow *(p + 3)$

```
int a[5];  
int *p = a
```

} $a[3] \Leftrightarrow *(p + 3)$
 $p[3] \Leftrightarrow *(a + 3)$



Representação de estruturas homogêneas de dados

- **Arrays Multidimensionais:**
 - Mesma forma de alocação e acesso a elementos;

elementos contíguos
na memória

$$\text{end}_{a[i]} = \text{end}_a + i * \text{sizeof}(T)$$

int **a[3][2]**

- **a** é um array de 3 elementos
 - **a[0]**, **a[1]** e **a[2]** são arrays de 2 inteiros

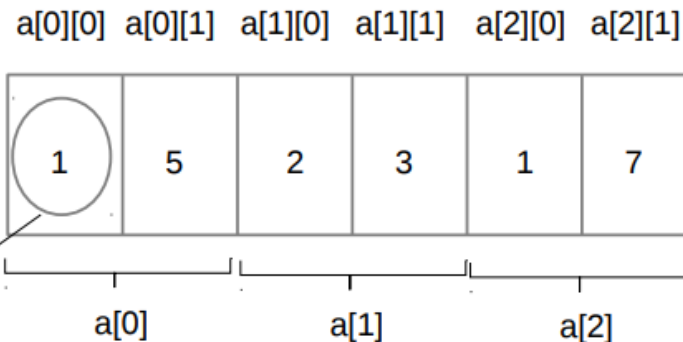


Representação de estruturas homogêneas de dados

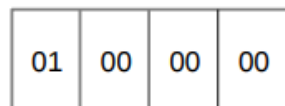
- **Arrays Multidimensionais:**

```
int a[3][2] = {{1,5},{2,3},{1,7}};
```

- **a** é um array de 3 elementos
- **a[0]**, **a[1]** e **a[2]** são arrays de 2 inteiros
- um inteiro tem 4 bytes



em little endian:

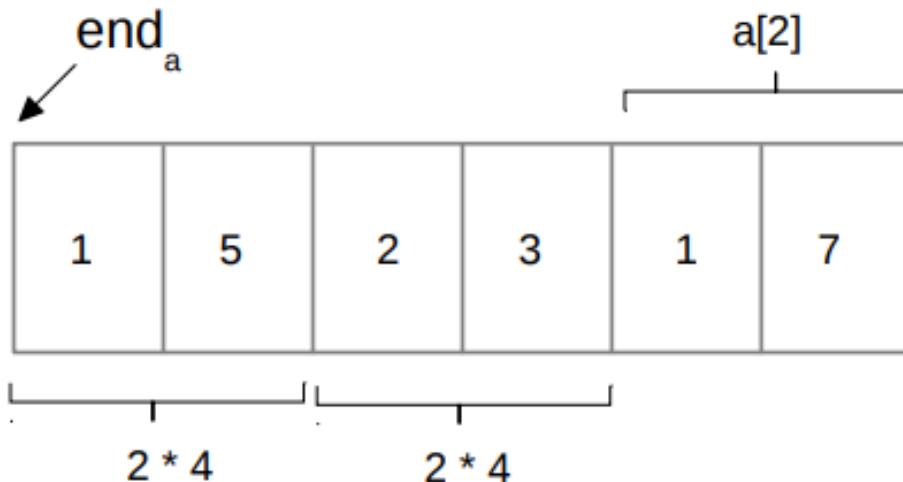


Representação de estruturas homogêneas de dados

- *Arrays Multidimensionais:*

```
int a[3][2] = {{1,5},{2,3},{1,7}};
```

$$\text{end}_{a[i]} \rightarrow \text{end}_a + i * \underbrace{2 * \text{sizeof}(\text{int})}_{\text{sizeof}(a[i])}$$



Representação de estruturas homogêneas de dados

- *Arrays Multidimensionais:*

```
int a[3][2] = {{1,5},{2,3},{1,7}};
```

- $\text{end}_{a[i]} \rightarrow \text{end}_a + i * \underbrace{2 * \text{sizeof}(\text{int})}_{\text{sizeof}(a[i])}$

- $\text{end}_{a[i][j]} \rightarrow \underbrace{\text{end}_a + i * 2 * \text{sizeof}(\text{int})}_{\text{end}_{a[i]}} + j * \text{sizeof}(\text{int})$



Representação de estruturas homogêneas de dados

1. Quando declaramos um parâmetro formal de uma função *C* que é um *array* unidimensional, não precisamos declarar seu tamanho. Por que?
1. Se o parâmetro for um *array* bidimensional, teremos que declarar pelo menos o número de "colunas". Por que?



Qual o tamanho (em bytes) de cada um dos *arrays* abaixo e como você acha que eles são armazenados em memória?

Exercícios

```
int b[2];  
short a[2][3];
```



Exercícios

1. Qual o tamanho (em bytes) de cada um dos *arrays* abaixo e como você acha que eles são armazenados em memória?

```
int b[2];  
short a[2][3];
```

```
#include <stdio.h>  
void dump (void *p, int n)  
{  
    unsigned char *p1 = p;  
    while (n--)  
    {  
        printf("%p - %02x\n", p1, *p1);  
        p1++;  
    }  
}  
int main ()  
{  
    int i,j;  
    short a[2][3];  
    int b[2];  
    for (i=0;i<2;i++)  
    {  
        b[i] = i;  
        for (j=0;j<3;j++)  
        {  
            a[i][j] = 3*i+j;  
        }  
    }  
    printf ("b: \n");  
    dump (b, sizeof(b));  
    printf ("a: \n");  
    dump (a, sizeof(a));  
    return 0;  
}
```

Exercícios

2. Analise o programa abaixo e faça contas para determinar o que será mostrado. Depois execute-o para conferir. Explique os valores mostrados.

```
#include <stdio.h>
void dump (void *p, int n)
{
    unsigned char *p1 = p;
    while (n--)
    {
        printf("%p - %02x\n", p1, *p1);
        p1++;
    }
}
int main ()
{
    short w = -32765;
    unsigned short k = 32771;
    printf("w=%d, k=%u \n", w, k);
    printf("dump de w: \n");
    dump(&w, sizeof(w));
    printf("dump de k: \n");
    dump(&k, sizeof(k));
    return 0;
}
```

Exercícios

3. Execute os programas abaixo (separadamente) e explique os resultados (pense na representação interna dos valores de x e y e na declaração de seus tipos).

```
/* programa 1*/
#include <stdio.h>
int main ()
{
    int x = 0xffffffff;
    int y = 2;
    printf("x=%d, y=%d\n", x, y);
    printf("x é menor do que y? %s\n", (x<y)?"sim":"nao");
    return 0;
}
```

```
/* programa 2*/
#include <stdio.h>
int main ()
{
    unsigned int x = 0xffffffff;
    unsigned int y = 2;
    printf("x=%d, y=%d\n", x, y);
    printf("x é menor do que y? %s\n", (x<y)?"sim":"nao");
    return 0;
}
```



Representação de estruturas de dados

- Muitos sistemas de computação impõem **restrições aos endereços** utilizados para acessar tipos de dados primitivos, exigindo que o endereço do objeto seja um múltiplo de algum valor K (normalmente 2, 4 ou 8).
- As restrições de alinhamento simplificam arquitetura do *hardware* que forma a interface entre o processador e o sistema de memória.



Representação de estruturas de dados

- Por exemplo, suponha que um processador sempre busque **8 bytes de memória** com um endereço que deve ser múltiplo de 8.
- Se pudermos garantir que qualquer **double** será alinhado para que seu endereço seja um múltiplo de 8, então o valor pode ser lido ou escrito com uma única operação de memória.
- Caso contrário, podemos precisar realizar dois acessos à memória, pois o objeto pode ser dividido em dois blocos de memória.



Representação de estruturas de dados

- A Intel (IA32) recomenda que os dados sejam alinhados para melhorar o sistema de memória desempenho.
- O Linux segue uma política de alinhamento em que tipos de dados de 2 *bytes* (por exemplo, short) deve ter um endereço que seja múltiplo de 2, enquanto qualquer tipo de dados maior (por exemplo, int, int *, float e double) deve ter um endereço que seja múltiplo de 4, 8,



Representação de estruturas de dados

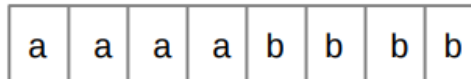
- Observe que esse requisito significa que o *bit* menos significativo do **endereço** de um objeto do tipo curto deve ser igual a zero.
- Da mesma forma, qualquer objeto do tipo int, ou qualquer ponteiro, deve estar em um endereço com os 2 *bits* de baixa ordem iguais a zero.



Representação de estruturas heterogêneas de dados

- Os campos de uma *struct* são alocados na memória sequencialmente, porém nem sempre contíguos

```
struct T1  
{  
    int a;  
    int b;  
};
```



layout de struct na memória depende da questão de **alinhamento**

```
struct T2  
{  
    int a;  
    char b;  
    int c;  
};
```



Representação de estruturas heterogêneas de dados

- Em plataformas de 64 *bits*, dados escalares de tamanho igual a *k bytes* devem ser alocados em endereços múltiplos de *k*:
 - **longs** em múltiplos de 8, **ints** em múltiplos de 4, **shorts** em múltiplos de 2.

```
struct T2
{
    int a;
    char b;
    int c;
};
```

bytes não usados são denominados padding



endereços múltiplos de 4



Representação de estruturas heterogêneas de dados

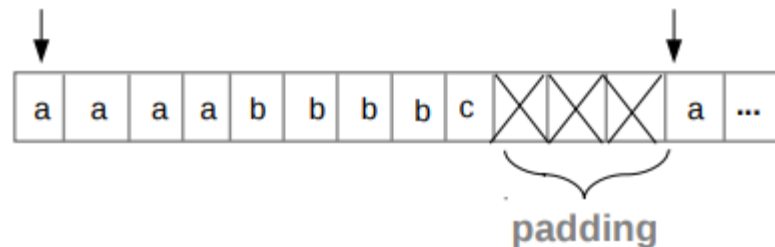
- O tamanho (*sizeof*) de uma estrutura deve garantir o alinhamento em um *array* de estruturas

```
struct T2
{
    int a;
    int b;
    char c;
};

int main()
{
    struct T2 v[2];

    return 0;
}
```

início de cada estrutura deve estar alinhado



Representação de estruturas heterogêneas de dados

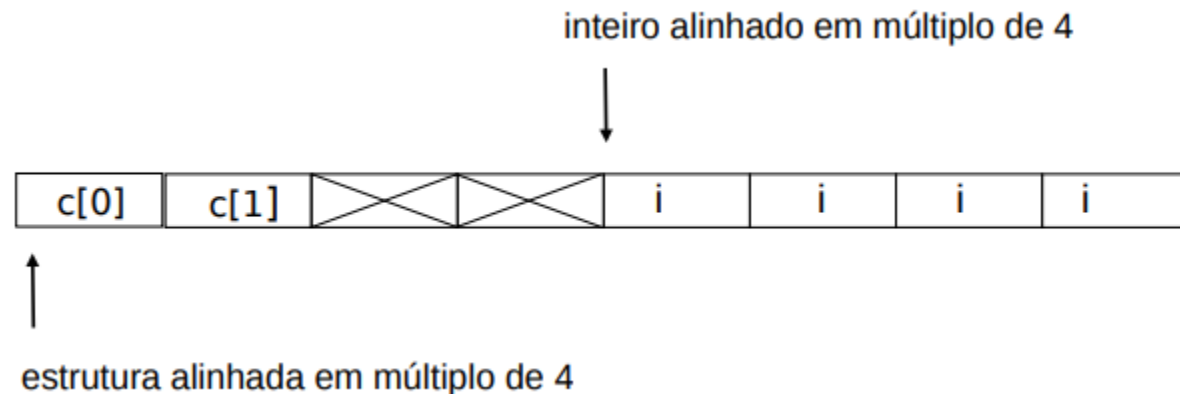
- O tamanho (*sizeof*) de uma estrutura deve garantir o alinhamento em um *array* de estruturas

```
struct T3
{
    char c[2];
    int i;
};
```

```
int main()
{
    struct T3 x;

    printf("x: %d\n", sizeof(x));
    /* sizeof(x) = 8 */

    return 0;
}
```



Representação de estruturas heterogêneas de dados

- Exemplos:

```
#include <stdio.h>
```

```
struct T4
```

```
{  
    short c,d;  
    char e;  
    short f;  
};
```

```
struct T5
```

```
{  
    int a;  
    long long b;  
};
```

```
int main()
```

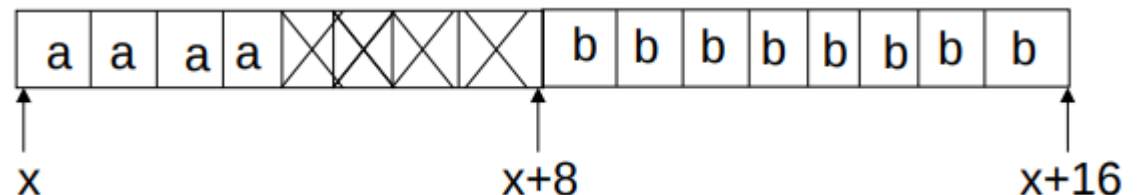
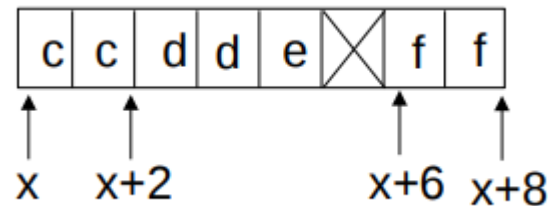
```
{  
    struct T4 x;  
    struct T5 y;
```

```
    printf("x: %d\n", sizeof(x));  
    /* sizeof(x) = 8 */
```

```
    printf("y: %d\n", sizeof(y));  
    /* sizeof(y) = 16 */
```

```
    return 0;
```

```
}
```



Exercícios

4. Analise e determine quantos bytes a estrutura heterogêneo de dados abaixo ocupa na memória principal. Depois use a função ***dump*** para ver como a estrutura abaixo é armazenada:

Experimente inicializá-la com valores facilmente "reconhecíveis" na memória, como, por exemplo:

```
typedef struct t1 T1;
struct t1
{
    int a;
    short b;
    int c;
};

int main()
{
    T1 s = {0xA1A2A3A4, 0xB1B2, 0xC1C2C3C4};

    return 0;
}
```



Exercícios

5. Analise e determine quantos bytes a estrutura heterogêneo de dados abaixo ocupa na memória principal. Depois use a função ***dump*** para ver como a estrutura abaixo é armazenada:

```
typedef struct t1 T1;  
struct t1  
{  
    char c1;  
    int i;  
    char c2;  
};
```

```
typedef struct t2 T2;  
struct t2  
{  
    long l;  
    char c;  
};
```

```
typedef struct t3 T3;  
struct t3  
{  
    int i;  
    char c1;  
    char c2;  
};
```



Exercícios

6. Analise e determine quantos bytes a estrutura heterogêneo de dados abaixo ocupa na memória principal. Depois use a função ***dump*** para ver como a estrutura abaixo é armazenada:

```
typedef struct t4 T4;  
struct t4  
{  
    struct X2 x;  
    char c;  
};
```

```
typedef struct t5 T5;  
struct t5  
{  
    char c1;  
    char c2;  
    char c3;  
};
```

```
typedef struct t6 T6;  
struct t6  
{  
    short s1;  
    int i;  
    char c[3];  
    short s2;  
};
```



Referência

